Hyper parameters are settings or configurations of a machine learning model that are tuned by the practitioner to optimize the model's performance. Here are examples of hyperparameters for various machine-learning models:

Lasso Regression:

'alpha' : [10,20,30,40,50] # Reduce overfitting and perform feature selection

Ridge Regression:

'alpha' : [10,20,30,40,50] # Reduce overfitting and handle multicollinearity

Elastic Net Regression:

'alpha' : [10,20,30,40,50] # Controls the overall regularization strength
'l1_ratio' : [0.1,0.3,0.5,0.7,0.9] # Controls the balance between L1 (Lasso) and L2 (Ridge) regularization

Logistic Regression:

'C' : [0.1, 1.0, 10.0], # Regularization parameter
'penalty' : ['l1', 'l2'], # Penalty type
'max_iter' : [100, 200, 300], # Maximum number of iterations

Support Vector Machines (SVC):

```
'C'          : [0.1, 1, 10, 100]                    # Controls regularization and the trade-off between margin size and classification errors
'kernel'     : ['linear', 'poly', 'rbf', 'sigmoid'] # Specifies the type of decision boundary transformation
'gamma'   : ['scale', 'auto']                       # Controls the influence/radius of individual training samples
'degree'    : [2, 3, 4]                             # Specifies the polynomial degree when kernel='poly'
'coefo'      : [0, 1]                               # Controls the independent term for polynomial and sigmoid kernels
```

Support Vector Machines (SVR):

```
'C'          : [0.1, 1, 10, 100]                    # Controls regularization and the penalty for errors outside the margin
'kernel'     : ['linear', 'poly', 'rbf', 'sigmoid'] # Specifies the type of function used to model nonlinear relationships
'gamma'   : ['scale', 'auto']                       # Controls the influence/radius of individual training samples
'degree'    : [2, 3, 4]                             # Specifies the polynomial degree when kernel='poly'
'coefo'      : [0, 1]                               # Controls the independent term for polynomial and sigmoid kernels
```

Decision Trees:

```
'criterion'         :  ['gini', 'entropy'],  # Criterion used for measuring the quality of a split
'max_depth'         :  [None, 5, 10, 15],    # Maximum depth of the trees
'min_samples_split' :  [2, 5, 10],           # Minimum number of samples required to split an internal node
'min_samples_leaf'  :  [1, 2, 4],            # Minimum number of samples required to be at a leaf node
'max_features'      :  ['auto', 'sqrt', 'log2'], # Number of features to consider when looking for the best split
'random_state'      :   [42]                 # Seed for the random number generator (for reproducibility)
```

Random Forests:

```
'criterion'         : ['gini', 'entropy'],    # Criterion used for measuring the quality of a split
'n_estimators'      : [50, 100, 200],         # Number of trees in the forest
'max_depth'         : [None, 5, 10],          # Maximum depth of the trees
'min_samples_split' : [2, 5, 10],             # Minimum number of samples required to split an internal node
'min_samples_leaf'  : [1, 2, 4],              # Minimum number of samples required to be at a leaf node
'max_features'      : ['auto', 'sqrt', 'log2'],     # Number of features to consider when looking for the best split
'random_state'      : [42]                    # Seed for the random number generator (for reproducibility)
```